

REGISTRO DE CO-CRIAÇÃO E DIRETRIZES TÉCNICAS: CONCEPÇÃO DO PORTFOLIOHUB COM SUPORTE DE IA

Estudante: Gabriel de Oliveira Sá Ferreira | **Assistente de Engenharia:** Google GEMINI | **Fase Acadêmica:** Entrega Final Consolidada

Apresentação do Documento: Este documento possui caráter duplo. Na primeira seção, detalha-se o histórico cronológico de engenharia e refinamento de requisitos (Logs de Prompting), demonstrando a interação ativa entre o estudante e a inteligência artificial para o direcionamento das tomadas de decisão. A segunda seção constitui o **Plano Técnico de Implementação Final** para a transição estrutural rumo ao ecossistema **PortfolioHUB**, em estrita conformidade com as exigências acadêmicas.

SEÇÃO 1: MEMORIAL TÉCNICO E HISTÓRICO DE CO-CRIAÇÃO (LOGS)

O processo de desenvolvimento do planejamento técnico baseou-se em ciclos iterativos de refinamento orientados pelas instruções da disciplina de Bootcamp 1. Abaixo constam os principais marcos e diálogos desta construção conjunta.

LOG #01: ALINHAMENTO DE CONTEXTO E ANÁLISE DA VERSÃO INTERMEDIÁRIA

[ESTUDANTE]: Boa noite gemini, estou no desenvolvimento de uma entrega final da matéria de bootcamp 1. Curso ciências da computação e queria que você fosse meu assistente durante isso. Primeiro irei mandar os links da entrega inicial e intermediária para discutirmos a final. Links: [Google Doc Currículo, Apresentação de Projetos, GitHub Page Mk3]

[GOOGLE GEMINI]: Recebi e examinei os documentos do seu ambiente. Identifiquei duas frentes técnicas robustas criadas por você: a Calculadora em Assembly (Marie.js) focada em lógica de hardware e mapeamento de display 16x16, e a Lista de 70 Exercícios de Lógica em Java Core. Sua GitHub Page (Mk3) atua como um agregador consolidando sua biografia de modding e aspirações como Dev Full Stack e Cientista de Dados. Para a entrega final, precisamos migrar essa base estática para um modelo de governança focado em mercado profissional.

LOG #02: DEFINIÇÃO DO ESCOPO E REGRAS DA ATIVIDADE

[ESTUDANTE]: O bootcamp 1 é para eu desenvolver e apresentar meus projetos pessoais e profissionais visando um mercado. Iremos fazer planejamentos de implementações, não elas em si. Esse é o comando: "Este desafio guiará o estudante por meio do processo completo de

implantação do PortfolioHUB, garantindo que adquira habilidades práticas na integração do GitHub e segurança, enquanto utilizam o Google GEMINI como apoio na implementação."

[GOOGLE GEMINI]: Compreendido perfeitamente. O foco transmuta-se de codificação direta para **arquitetura de planejamento, governança e conformidade SecOps**. Alinhando seu comando às diretrizes, estruturamos uma estratégia dividida em 4 fases sequenciais bem delineadas: Gestão de Usuários no GitHub, Políticas de Segurança Assistidas por IA, Documentação/Colaboração de Padrão Aberto e Preparação para Produção (Lançamento Real).

LOG #03: REFINAMENTO ITERATIVO POR FASES (FASES 1 A 4)

[ESTUDANTE]: Será estritamente individual. [Comandos para ativação e sequenciamento das Fases 1, 2, 3 e 4].

[GOOGLE GEMINI]: Sendo individual, focamos na autoproteção do engenheiro contra erros operacionais comuns. Mapeamos sequencialmente: proteção de branches via restrição de push direto e exigência de Pull Requests (Fase 1); DevSecOps passivo com Dependabot, Secret Scanning e auditoria pré-commit via Gemini contra riscos OWASP (Fase 2); padronização através de Conventional Commits e arquivos de governança corporativa como templates de PR e CONTRIBUTING.md (Fase 3); e automação do pipeline final de CI/CD via GitHub Actions integrado ao ecossistema do GitHub Pages com auditoria de links quebrados da versão Mk3 (Fase 4).

SEÇÃO 2: PLANO TÉCNICO DE IMPLEMENTAÇÃO (ENTREGA FINAL)

Abaixo apresenta-se o artefato técnico consolidado e revisado, gerado a partir do processo de mentoria assistida documentado na seção anterior.

FASE 1: GESTÃO DE USUÁRIOS E CONTROLE DE ACESSO (GITHUB)

A adoção de políticas rigorosas de governança de identidade e de ramificação de código (branching) é indispensável para certificar a maturidade técnica exigida por grandes corporações.

1.1 Governança da Identidade e Autenticação

- **Autenticação em Dois Fatores (2FA):** Ativação compulsória de 2FA na conta institucional/profissional do GitHub via aplicativos autenticadores baseados em tempo (TOTP). Essa medida mitiga o risco de sequestro de credenciais e alterações maliciosas no portfólio de produção.
- **Segurança na Linha de Comando (CLI):** Fica estabelecido o uso exclusivo de chaves criptográficas **SSH (Ed25519)** ou **Personal Access Tokens (PAT)** com escopos limitados para operações de push, eliminando chaves estáticas em terminal.

1.2 Arquitetura de Branches (Fluxo de Trabalho Git)

O desenvolvimento abandonará a prática de commits diretos na ramificação principal, adotando um fluxo baseado em isolamento de escopo:

Branch	Função Estratégica	Regra Operacional
<code>main</code>	Ambiente de Produção	Contém apenas o código estável, auditado e renderizado na GitHub Page pública.
<code>feature/*</code>	Novas Funcionalidades	Ramificações temporárias criadas para modificações específicas (ex: <code>feature/secao-dados</code>).
<code>fix/*</code>	Correção de Bugs	Ramificações exclusivas para debugar problemas identificados em produção.

1.3 Regras de Proteção de Branch (Branch Protection Rules)

Para atuar como uma salvaguarda contra erros operacionais do próprio desenvolvedor, serão configuradas as seguintes regras na interface administrativa do GitHub para a branch `main` :

- **Exigência de Pull Request (PR):** Bloqueio total de pushes diretos do terminal para a ``main``. Toda alteração deve passar pelo processo formal de abertura de Pull Request no painel web.
- **Bloqueio de Forçar Push (Force Push):** Desativação completa de comandos destrutivos como `git push --force` , blindando a integridade histórica dos commits do projeto.

FASE 2: POLÍTICAS DE SEGURANÇA COM APOIO DO GEMINI

A segurança da informação será integrada diretamente no ciclo de vida do desenvolvimento (DevSecOps) utilizando mecanismos automatizados do GitHub em sinergia com o Google Gemini.

2.1 Gestão de Segredos e Dados Sensíveis (Secret Management)

- **Uso Rigoroso do `.gitignore`**: Mapeamento de um arquivo de exclusão robusto na raiz do repositório para impedir o rastreamento acidental de artefatos de IDEs (`.vscode/` , `.idea/`) ou arquivos temporários do sistema operacional (`.DS_Store`).
- **Abstração de Credenciais**: Armazenamento de tokens ou chaves de consumo de APIs externas no recurso criptografado **GitHub Secrets**, injetando-os dinamicamente em tempo de execução sem exposição no código aberto.

2.2 Automação de Varreduras de Segurança

- **Dependabot Alerts**: Ativação do robô nativo do GitHub para monitorar continuamente as dependências associadas aos subprojetos (como pacotes e bibliotecas Java).
- **GitHub Secret Scanning**: Habilitação da varredura passiva de histórico. Se qualquer string interpretada como token ou senha for detectada, o GitHub efetuará o bloqueio imediato da exposição.

Diretriz SecOps - Atuação do Google Gemini:

Durante a fase de codificação de novas funcionalidades, o Gemini será utilizado de forma consultiva para realizar revisões de código pré-commit. O estudante submeterá blocos de código (HTML/Java) à IA para validação contra falhas clássicas de segurança da OWASP.

FASE 3: DOCUMENTAÇÃO E PRÁTICAS DE COLABORAÇÃO

A clareza técnica e a rastreabilidade das alterações são métricas de alta relevância na avaliação de portfólios pelo mercado técnico.

3.1 Estrutura de Documentação de Repositório (README.md)

O repositório principal do PortfolioHUB e os repositórios modulares de projetos conterão arquivos de documentação estruturados nos seguintes tópicos obrigatórios: Pitch do Projeto, Arquitetura Técnica, Guia de Execução Local e Roadmap de Evolução.

3.2 Padronização de Histórico (Conventional Commits)

As mensagens de commit seguirão o padrão internacional de legibilidade:

- **feat:** Introdução de novas funcionalidades (ex: `feat: adicionar meta tags de SEO`).
- **fix:** Correção de falhas ou bugs (ex: `fix: corrigir caminho relativo da imagem da faculdade`).
- **docs:** Atualizações exclusivas de documentação (ex: `docs: expandir seção de instalação local`).

3.3 Governança Avançada: Templates de Repositório

- **Pull Request Template (`pull_request_template.md`)**: Checklist obrigatório auto-aplicável para ser preenchido antes de consolidar a branch, verificando testes locais e segurança.
- **Arquivo CONTRIBUTING.md**: Documento norteador com as boas práticas para que avaliadores externos possam submeter melhorias ao projeto de forma padronizada.

| FASE 4: PREPARAÇÃO PARA PRODUÇÃO E LANÇAMENTO REAL

A fase final garante a qualidade do produto antes do deploy definitivo no servidor de hospedagem do GitHub Pages.

4.1 Resolução de Débitos Técnicos da Versão Intermediária

- **Corretiva de Mídias:** Resolução definitiva da imagem ausente identificada na entrega intermediária (`img/facul.jpg não encontrado`).
- **Sanitização de Hiperlinks:** Auditoria para garantir que todos os links externos de direcionamento (LinkedIn e repositórios) funcionem perfeitamente com a tag de isolamento `target="_blank"` .

4.2 Otimização de Performance, SEO e Deploy Contínuo (CI/CD)

As mídias serão convertidas para o formato **WebP** visando tempo mínimo de carregamento. O HTML contará com tags Open Graph para gerar cards de pré-visualização no LinkedIn.

A publicação manual de arquivos será substituída por automação contínua utilizando **GitHub Actions**. Ao aprovar um Pull Request na branch principal, o script efetuará automaticamente o build limpo, testes de sintaxe e atualização dos servidores do GitHub Pages.